

# PRÁCTICA 3

TÉCNICAS DE BÚSQUEDA DE POBLACIONES PARA EL PROBLEMA  
DE ASIGNACIÓN CON RESTRICCIONES (PAR)



# UNIVERSIDAD DE GRANADA

Metaheurísticas

Grupo A2: Viernes 15:30 - Curso 2025-2026

Jorge García Gámiz

DNI: 77449859G

e-mail: [jorgegarciag@correo.ugr.es](mailto:jorgegarciag@correo.ugr.es)

# Índice

|                                                                           |           |
|---------------------------------------------------------------------------|-----------|
| <b>1. Descripción del Problema</b>                                        | <b>2</b>  |
| <b>2. Aplicación de los algoritmos al problema</b>                        | <b>3</b>  |
| <b>3. Estructura de los algoritmos</b>                                    | <b>9</b>  |
| 3.1. Enfriamiento Simulado (ES)                                           | 9         |
| 3.2. Búsqueda Multiarranque Básica (BMB)                                  | 9         |
| 3.3. Búsqueda Local Reiterada (ILS)                                       | 11        |
| 3.4. ILS-ES (Hibridación de ILS y ES)                                     | 12        |
| 3.5. GRASP (variante voluntaria)                                          | 12        |
| 3.6. ES-Fast (variante voluntaria)                                        | 14        |
| <b>4. Estructura del código</b>                                           | <b>15</b> |
| 4.1. Manual de compilación                                                | 15        |
| 4.2. Manual de ejecución                                                  | 16        |
| <b>5. Experimentación y Análisis de Resultados</b>                        | <b>17</b> |
| 5.1. Descripción de los casos de prueba                                   | 17        |
| 5.2. Parámetros de ejecución                                              | 17        |
| 5.3. Resultados obtenidos                                                 | 18        |
| 5.4. Análisis de resultados                                               | 20        |
| 5.4.1. Comparación general entre algoritmos                               | 21        |
| 5.4.2. Impacto de las restricciones en la función objetivo                | 21        |
| 5.4.3. Esfuerzo computacional y convergencia                              | 22        |
| 5.4.4. Comparación entre conjuntos de datos y dimensionalidad             | 22        |
| 5.4.5. Análisis de la variabilidad de los resultados                      | 22        |
| 5.4.6. Eficiencia computacional y tiempos de ejecución                    | 29        |
| 5.4.7. Conclusiones generales del análisis                                | 29        |
| <b>6. Voluntario: Contraste de Hipótesis mediante el Test de Wilcoxon</b> | <b>32</b> |
| <b>7. Referencias y recursos utilizados</b>                               | <b>34</b> |

# 1. Descripción del Problema

El Problema de Asignación con Restricciones (PAR) es una generalización del problema de agrupamiento clásico que busca particionar un conjunto de  $n$  instancias en  $k$  clusters disjuntos, donde cada instancia  $\vec{x}_i \in \mathbb{R}^d$  tiene  $d$  características numéricas.

Además, se dispone de una matriz de restricciones  $R \in M_{n \times n}$ , donde:

- $R_{ij} = 1$  es Must-Link (ML):  $i$  y  $j$  deben pertenecer al mismo cluster
- $R_{ij} = -1$  es Cannot-Link (CL):  $i$  y  $j$  deben estar en distintos clusters
- $R_{ij} = 0$  indica ausencia de restricción.

Una solución se representa como un vector de asignación  $s = (s_1, \dots, s_n)$  con  $s_i \in \{0, \dots, k-1\}$  asigna cada instancia a un cluster.

## Función Objetivo

El objetivo es obtener una partición solución que permita minimizar la función fitness definida como:

$$fitness(s) = desviacion(s) + \lambda \cdot infeasibility(s), \quad (1)$$

donde el parámetro  $\lambda = \frac{\max_{i,j} \|\vec{x}_i - \vec{x}_j\|}{R} \in [0, 1[$  equilibra ambos términos ( $R$  es el número de restricciones).

## Desviación

La calidad del clustering se mide mediante la desviación general. Primero se calculan los centroides. Para cada cluster  $C_i$ , con  $i \in \{0, \dots, k-1\}$ , su centroide es:

$$\vec{\mu}_i = \frac{1}{|C_i|} \sum_{\vec{x}_j \in C_i} \vec{x}_j$$

Luego la distancia media intra-cluster, que es la media de las distancias de las instancias que lo conforman a su centroide:

$$dist_{intra-cluster}(C_i) = \frac{1}{|C_i|} \sum_{\vec{x}_j \in C_i} \|\vec{x}_j - \vec{\mu}_i\|$$

Definimos la desviación general de la partición en clusters  $C = \{C_1, \dots, C_k\}$  como la media de las desviaciones intra-cluster de cada cluster:

$$desviacion(C) = \frac{1}{k} \sum_{i=1}^k dist_{intra-cluster}(C_i)$$

## Infeasibility

La “infeasibility” cuenta el número de restricciones violadas:

$$infeasibility = \sum_{(i,j) \in ML} [s_i \neq s_j] + \sum_{(i,j) \in CL} [s_i = s_j]$$

## 2. Aplicación de los algoritmos al problema

En esta práctica se han implementado cuatro algoritmos de búsqueda basados en trayectorias para la resolución del PAR: Enfriamiento Simulado (ES), Búsqueda Multiarranque Básica (BMB), Búsqueda Local Reiterada (ILS) e ILS-ES (hibridación de ILS y ES). Adicionalmente, como trabajo voluntario, se han implementado dos variantes extra: GRASP y una versión rápida del ES con evaluación incremental de la infeasibility (ES-Fast). Todos ellos heredan de la clase base `MH<int>`, implementando el método `optimize()` de la interfaz, lo que permite integrarlos en el marco de algoritmos existente sin modificar el diseño original.

### Representación del problema

El problema se encapsula en la clase `ProblemPAR`, encargada de almacenar los datos de la instancia y proporcionar los métodos necesarios para evaluar soluciones. Esta clase mantiene como atributos principales:

- `features`: matriz que contiene las características de cada instancia del conjunto de datos.
- `constraints`: matriz de restricciones que indica las relaciones Must-Link y Cannot-Link entre pares de instancias.
- `num_clusters`: número de clusters en los que se debe dividir el conjunto de instancias.
- `lambda`: parámetro de penalización utilizado en la función objetivo.

Durante la construcción del problema se leen los datos desde fichero y se calcula automáticamente el valor de  $\lambda$  según la expresión definida en la sección anterior.

### Cálculo de centroides

Para evaluar la calidad de una solución es necesario calcular los centroides de los clusters definidos por dicha solución. El procedimiento utilizado es el mostrado en la siguiente página.

### Cálculo de la desviación

Una vez obtenidos los centroides, se calcula la desviación media intra-cluster de cada cluster y posteriormente la desviación global de la partición.

---

```
function CALCULATECENTROIDS(solution)
  num_instancias  $\leftarrow$  longitud(features)
  num_caracteristicas  $\leftarrow$  longitud(features[0])
  Inicializar centroides[k][num_caracteristicas] con 0,0, contador[k] con 0
  for i  $\leftarrow$  0 to num_instancias - 1 do
    c  $\leftarrow$  solution[i]
    if c  $\geq$  0 and c < k then
      for j  $\leftarrow$  0 to num_caracteristicas - 1 do
        centroides[c][j]  $\leftarrow$  centroides[c][j] + features[i][j]
      end for
      contador[c]  $\leftarrow$  contador[c] + 1
    end if
  end for
  for c  $\leftarrow$  0 to k - 1 do
    if contador[c] > 0 then
      for j  $\leftarrow$  0 to num_caracteristicas - 1 do
        centroides[c][j]  $\leftarrow$  centroides[c][j]/contador[c]
      end for
    end if
  end for
  return centroides
end function
```

---

---

```
function CALCULATEDEVIATION(solution)
  centroides  $\leftarrow$  CALCULATECENTROIDS(solution)
  num_instancias  $\leftarrow$  longitud(features)
  Inicializar dist_cluster[k] con 0,0, contador[k] con 0
  for i  $\leftarrow$  0 to num_instancias - 1 do
    c  $\leftarrow$  solution[i]
    if c = -1 then
      continue
    end if
    dist_cluster[c]  $\leftarrow$  dist_cluster[c] + distancia(features[i], centroides[c])
    contador[c]  $\leftarrow$  contador[c] + 1
  end for
  total_dev  $\leftarrow$  0,0
  for c  $\leftarrow$  0 to k - 1 do
    if contador[c] > 0 then
      desv  $\leftarrow$  dist_cluster[c]/contador[c]
    else
      desv  $\leftarrow$  999999,0 ▷ penalización por cluster vacío
    end if
    total_dev  $\leftarrow$  total_dev + desv
  end for
  return total_dev/k
end function
```

---

Donde en distancia estamos usando la distancia euclídea implementada en una función `calculateEuclideanDistance(instancia i1, instancia i2)`.

Si algún cluster queda vacío, se aplica una penalización elevada (999999.0) para evitar que este tipo de soluciones sea favorecido por los algoritmos de búsqueda.

### Cálculo de infeasibility

El término de infeasibility mide el número de restricciones violadas por una solución. Para ello se recorren todos los pares de instancias y se comprueba si la asignación de clusters respeta las restricciones Must-Link y Cannot-Link.

---

```
function CALCULATEINFEASIBILITY(solution)
    infeas  $\leftarrow$  0
    n  $\leftarrow$  longitud(features)
    for i  $\leftarrow$  0 to n - 1 do
        if solution[i] = -1 then
            continue
        end if
        for j  $\leftarrow$  i + 1 to n - 1 do
            if solution[j] = -1 then
                continue
            end if
            res  $\leftarrow$  constraints[i][j]
            if res  $\neq$  0 then
                mismo_cluster  $\leftarrow$  (solution[i] = solution[j])
                if (res = 1 and not mismo_cluster) or (res = -1 and
mismo_cluster) then
                    infeas  $\leftarrow$  infeas + 1
                end if
            end if
        end for
    end for
    return infeas
end function
```

---

### Función Objetivo

Finalmente, el valor de fitness de una solución se calcula (véase Ecuación (1)) combinando la desviación intra-cluster y el número de restricciones violadas mediante el parámetro  $\lambda$  descrito anteriormente. Así se refleja en el código:

---

```
function FITNESS(solution)
    desviacion  $\leftarrow$  CALCULATEDEVIAION(solution)
    infeas  $\leftarrow$  CALCULATEINFEASIBILITY(solution)
     $\lambda \leftarrow$  CALCULATELAMBDA()  $\triangleright$  Se asume que  $\lambda$  se obtiene de la función
    return desviacion +  $\lambda \cdot$  infeas
end function
```

---

Esta función permite equilibrar la calidad del agrupamiento con el grado de cumplimiento de las restricciones, penalizando aquellas soluciones que violan un mayor número de ellas.

Por su parte, para calcular  $\lambda$  (*lambda*) en (1), usamos la función siguiente:

---

```
function CALCULATELAMBDA
    max_dist  $\leftarrow$  0  $\triangleright$  Máxima distancia entre pares
    for all pares (i, j) con i < j do
        dist  $\leftarrow$  distancia(features[i], features[j])
        if dist > max_dist then
            max_dist  $\leftarrow$  dist
        end if
    end for
    R  $\leftarrow$  0  $\triangleright$  Contar restricciones
    for all pares (i, j) con i < j do
        if constraints[i][j]  $\neq$  0 then
            R  $\leftarrow$  R + 1
        end if
    end for
    if R = 0 then
        return 0
    end if
    return  $\frac{\textit{max\_dist}}{R}$ 
end function
```

---

## Generación de solución inicial válida

Todos los algoritmos implementados parten de una solución inicial aleatoria. Sin embargo, la generación puramente aleatoria puede producir clusters vacíos, lo que invalida la solución (la función de desviación asignaría una penalización artificial de 999999 a dichos clusters).

Para garantizar la validez, el método `createSolution()` de la clase `ProblemPAR` fue corregido respecto a la práctica anterior para seguir la siguiente estrategia: primero se barajan aleatoriamente todos los índices de instancias; a continuación, se asigna de forma forzada una instancia a cada uno de los  $k$  clusters; finalmente, el resto de instancias se distribuyen de manera uniforme aleatoria entre los  $k$  clusters disponibles. Este procedimiento garantiza que toda solución generada sea válida según el método `isValid()`, que en esta práctica también ha sido reforzado para

comprobar no solo que los valores estén en el rango  $[0, k - 1]$ , sino también que ningún cluster quede vacío.

---

```
function CREATESOLUTION
   $n \leftarrow \text{longitud}(\text{features})$ 
   $s \leftarrow$  vector de tamaño  $n$  inicializado a  $-1$ 
   $\text{available} \leftarrow [0, 1, \dots, n - 1]$  barajado aleatoriamente
  for  $c \leftarrow 0$  hasta  $k - 1$  do
     $s[\text{available}[c]] \leftarrow c$  ▷ Un elemento forzado por cluster
  end for
  for  $i \leftarrow k$  hasta  $n - 1$  do
     $s[\text{available}[i]] \leftarrow$  entero aleatorio en  $[0, k - 1]$ 
  end for
  return  $s$ 
end function
```

---

Los algoritmos que requieren partir de una solución externa (ILS, ILS-ES, GRASP) generan su solución inicial con este mismo patrón de forma inline, sin delegar en `createSolution()`, ya que necesitan acceso directo al vector resultante antes de pasarlo al optimizador interno. `LocalSearch` en su modo sin argumento sí llama directamente a `createSolution()`. **Por legibilidad, en los pseudocódigos de los algoritmos de la siguiente sección se referencia `createSolution()` siempre que se necesita una solución aleatoria inicial válida.**

### Búsqueda Local (BL) como componente base

La BL de la Práctica 1 constituye el núcleo de intensificación de todos los algoritmos de trayectoria múltiple.

El operador de vecino consiste en reasignar una instancia a un cluster distinto del actual. Para explorar el vecindario sin sesgo de orden, se construye la lista completa de pares (instancia, cluster destino) y se baraja aleatoriamente en cada iteración. Se sigue una estrategia *first-improvement*: al encontrar el primer movimiento estrictamente mejor, se acepta y se vuelve a generar el vecindario. Además, se comprueba explícitamente la validez de cada vecino con `isValid()` antes de aceptarlo, lo que garantiza que ningún cluster quede vacío durante la búsqueda.

---

```
function FIX(solution)
  for  $i \leftarrow 0$  to  $\text{longitud}(\text{solution}) - 1$  do
    if  $\text{solution}[i] < 0$  then
       $\text{solution}[i] \leftarrow 0$ 
    end if
    if  $\text{solution}[i] \geq k$  then
       $\text{solution}[i] \leftarrow k - 1$ 
    end if
  end for
end function
```

---



---

```
function LOCALSEARCH(problem, maxevals, s0 (opcional))
  n ← problem.getSolutionSize()
  k ← problem.getSolutionDomainRange().second + 1
  if s0 no se proporciona then
    s ← problem.createSolution()                                ▷ Garantiza clusters no vacíos
  else
    s ← s0
  end if
  fs ← problem.fitness(s);  evals ← 1;  improvement ← true
  while improvement and evals < maxevals do
    improvement ← false
    N ← {(i, c) | c ≠ si} barajado aleatoriamente
    for cada (i, c') ∈ N do
      if evals ≥ maxevals then break
      end if
      if cluster_counts[si] ≤ 1 then continue
      end if
      s' ← s con s'i ← c'
      if ¬ problem.isValid(s') then continue
      end if
      fs' ← problem.fitness(s');  evals ← evals + 1
      if fs' < fs then
        s ← s';  fs ← fs';  improvement ← true;  break
      end if
    end for
  end while
  return (s, fs, evals)
end function
```

---

---

```
function ISVALID(solution)
  Inicializar has_element[0...k - 1] con false
  for all val ∈ solution do
    if val < 0 or val ≥ k then
      return false
    end if
    has_element[val] ← true
  end for
  for c ← 0 to k - 1 do
    if not has_element[c] then
      return false
    end if
  end for
  return true
end function
```

---

### 3. Estructura de los algoritmos

1

#### 3.1. Enfriamiento Simulado (ES)

El ES es una metaheurística de trayectoria simple inspirada en el proceso de temple de metales. A diferencia de la BL, permite aceptar soluciones peores que la actual con una probabilidad que decrece con la temperatura  $T$ , lo que facilita escapar de óptimos locales en las fases iniciales de la búsqueda.

La temperatura inicial se calcula en función del coste de la solución de partida:  $T_0 = \mu \cdot f(s_0) / (-\ln \varphi)$ , con  $\mu = 0,2$  y  $\varphi = 0,3$ . Esto implica que al comienzo se acepta con probabilidad  $\varphi$  una solución un  $\mu$  por uno peor que la inicial. La temperatura final se fija en  $T_f = 10^{-3}$ ; si  $T_0 \leq T_f$ , se eleva  $T_0$  a  $10 \cdot T_f$  para garantizar un rango de enfriamiento válido.

El esquema de enfriamiento empleado es el de Cauchy modificado:

$$T_{k+1} = \frac{T_k}{1 + \beta \cdot T_k}, \quad \beta = \frac{T_0 - T_f}{M \cdot T_0 \cdot T_f}$$

donde  $M = \lfloor \text{maxevals} / \text{max\_vecinos} \rfloor$  es el número total de enfriamientos.

El bucle interno  $L(T)$  genera vecinos de uno en uno, de forma aleatoria y sin repetición dentro de cada paso (lista barajada que se recicla si se agota). La condición de terminación del bucle interno es doble: se para al alcanzar  $\text{max\_vecinos} = 10 \cdot n$  vecinos generados o  $\text{max\_exitos} = 1 \cdot n$  vecinos aceptados. El criterio de aceptación sigue la regla de Metrópolis: se acepta siempre si  $\Delta f < 0$ , y con probabilidad  $e^{-\Delta f/T}$  en caso contrario. El algoritmo mantiene la mejor solución global encontrada durante todo el proceso, que es la que finalmente se devuelve.

#### 3.2. Búsqueda Multiarranque Básica (BMB)

La BMB es el algoritmo de trayectoria múltiple más sencillo. Su idea es eliminar la dependencia de la solución inicial ejecutando la BL desde múltiples puntos de partida independientes, esperando que alguno de ellos se encuentre en la cuenca de atracción de un buen óptimo local.

En esta implementación se realizan 5 reinicios, asignando a cada uno un presupuesto de  $\lfloor \text{maxevals}/5 \rfloor = 20\,000$  evaluaciones. Cada reinicio genera su propia solución aleatoria inicial (internamente, mediante LOCALSEARCH) y aplica la BL completa sobre ella. Al finalizar todos los reinicios se devuelve la mejor solución encontrada en el conjunto.

---

<sup>1</sup>Pseudocódigos de ES y BMB en la siguiente página.

---

```

function SIMULATEDANNEALING(problem, maxevals)
   $n \leftarrow \text{problem.getSolutionSize}()$ 
   $k \leftarrow \text{problem.getSolutionDomainRange}().\text{second} + 1$ 
   $\text{max\_vecinos} \leftarrow 10n$ ;  $\text{max\_exitos} \leftarrow n$ ;  $M \leftarrow \lfloor \text{maxevals} / \text{max\_vecinos} \rfloor$ 
   $s \leftarrow \text{problem.createSolution}()$ ;  $f_s \leftarrow \text{problem.fitness}(s)$ ;  $\text{evals} \leftarrow 1$ 
   $T_0 \leftarrow (\mu \cdot f_s) / (-\ln \varphi)$ ; si  $T_0 \leq T_f$  entonces  $T_0 \leftarrow 10 T_f$ 
   $\beta \leftarrow (T_0 - T_f) / (M \cdot T_0 \cdot T_f)$ ;  $T \leftarrow T_0$ ;  $s^* \leftarrow s$ ;  $f^* \leftarrow f_s$ 
  while  $\text{evals} < \text{maxevals}$  do
     $n\_exitos \leftarrow 0$ ;  $n\_vecinos \leftarrow 0$ 
    Barajar orden de instancias  $\sigma$ ;  $\text{idx} \leftarrow 0$ 
    while  $n\_exitos < \text{max\_exitos}$  and  $n\_vecinos < \text{max\_vecinos}$  and  $\text{evals} < \text{maxevals}$  do
      if  $\text{idx} \geq n$  then Rebarajar  $\sigma$ ;  $\text{idx} \leftarrow 0$ 
      end if
       $i \leftarrow \sigma[\text{idx}]$ ;  $\text{idx} += 1$ 
      if  $\text{cluster\_counts}[s_i] \leq 1$  then continue
      end if
       $c' \leftarrow \text{cluster aleatorio} \neq s_i$ ;  $s' \leftarrow s$  con  $s'_i \leftarrow c'$ 
       $f_{s'} \leftarrow \text{problem.fitness}(s')$ ;  $\text{evals} += 1$ ;  $n\_vecinos += 1$ 
       $\Delta f \leftarrow f_{s'} - f_s$ 
      if  $\Delta f < 0$  or  $U(0, 1) \leq e^{-\Delta f / T}$  then
         $s \leftarrow s'$ ;  $f_s \leftarrow f_{s'}$ ;  $n\_exitos += 1$ 
        if  $f_s < f^*$  then  $s^* \leftarrow s$ ;  $f^* \leftarrow f_s$ 
        end if
      end if
    end while
     $T \leftarrow T / (1 + \beta \cdot T)$ 
    if  $n\_exitos = 0$  then break
    end if
  end while
  return ( $s^*$ ,  $f^*$ ,  $\text{evals}$ )
end function

```

---



---

```

function BMB(problem, maxevals)
   $\text{num\_restarts} \leftarrow 5$ ;  $\text{evals\_per\_restart} \leftarrow \lfloor \text{maxevals} / \text{num\_restarts} \rfloor$ 
   $s^* \leftarrow \emptyset$ ;  $f^* \leftarrow +\infty$ ;  $\text{evals\_total} \leftarrow 0$ 
  for  $i \leftarrow 1$  hasta  $\text{num\_restarts}$  do
    ( $s'$ ,  $f'$ ,  $e$ )  $\leftarrow \text{LOCALSEARCH}(\text{problem}, \text{evals\_per\_restart})$ 
     $\text{evals\_total} \leftarrow \text{evals\_total} + e$ 
    if  $f' < f^*$  then  $s^* \leftarrow s'$ ;  $f^* \leftarrow f'$ 
    end if
  end for
  return ( $s^*$ ,  $f^*$ ,  $\text{evals\_total}$ )
end function

```

---

### 3.3. Búsqueda Local Reiterada (ILS)

El ILS supera la limitación de la BMB añadiendo memoria entre reinicios: en lugar de partir siempre de una solución completamente nueva, aplica una mutación sobre la mejor solución encontrada hasta el momento. Así, cada reinicio no explora el espacio desde cero, sino desde un punto cercano (pero suficientemente distante) al mejor óptimo local hallado, favoreciendo la exploración de regiones prometedoras del espacio de búsqueda.

El criterio de aceptación seguido es el del *mejor*: la mutación se aplica siempre sobre la mejor solución global acumulada, independientemente de la calidad del último reinicio. El operador de mutación modifica el 20 % de los genes de la solución, asignando a cada uno un cluster distinto del original, seleccionado de forma uniforme aleatoria. Los índices a mutar se eligen sin repetición mediante barajado.

Al igual que BMB, se realizan 5 iteraciones con 20 000 evaluaciones cada una: la primera sobre una solución aleatoria y las cuatro restantes sobre versiones mutadas de la mejor solución acumulada.

---

```

function MUTATE( $s, k$ )
   $num\_to\_change \leftarrow \max(1, \lfloor 0,2 \cdot |s| \rfloor)$ ;  $s' \leftarrow s$ 
  Seleccionar  $num\_to\_change$  índices distintos al azar  $\rightarrow \mathcal{I}$ 
  for cada  $i \in \mathcal{I}$  do
     $s'_i \leftarrow$  cluster aleatorio  $\neq s'_i$ 
  end for
  return  $s'$ 
end function

```

---



---

```

function ILS( $problem, maxevals$ )
   $n \leftarrow problem.getSolutionSize()$ 
   $k \leftarrow problem.getSolutionDomainRange().second + 1$ 
   $num\_iters \leftarrow 5$ ;  $evals\_per\_restart \leftarrow \lfloor maxevals/num\_iters \rfloor$ 
   $s_0 \leftarrow problem.createSolution()$ 
   $(s^*, f^*, e) \leftarrow LOCALSEARCH(problem, evals\_per\_restart, s_0)$ 
   $evals\_total \leftarrow e$ 
  for  $i \leftarrow 1$  hasta  $num\_iters - 1$  do
     $s_m \leftarrow MUTATE(s^*, k)$  ▷ Siempre se muta la mejor solución global
     $(s', f', e) \leftarrow LOCALSEARCH(problem, evals\_per\_restart, s_m)$ 
     $evals\_total += e$ 
    if  $f' < f^*$  then  $s^* \leftarrow s'$ ;  $f^* \leftarrow f'$ 
  end if
  end for
  return  $(s^*, f^*, evals\_total)$ 
end function

```

---

### 3.4. ILS-ES (Hibridación de ILS y ES)

El ILS-ES mantiene la estructura externa del ILS (mutación sobre la mejor solución y criterio de aceptación del mejor) pero sustituye la BL como optimizador interno por el Enfriamiento Simulado. Esto combina la capacidad de diversificación del esquema iterado con la mayor potencia de intensificación del ES, que puede escapar de óptimos locales durante cada fase de exploración.

Los parámetros del ES interno son idénticos a los del ES independiente ( $\mu$ ,  $\varphi$ ,  $T_f$ , esquema de Cauchy), con la única diferencia de que la solución de partida de cada ejecución es la solución mutada, no una aleatoria. El operador de mutación utilizado es exactamente el mismo que en ILS (20 % de genes, reutilizado desde `ILS::mutate`).

---

```
function RUNES(problem, s0, maxevals)
  Idéntico a SIMULATEDANNEALING pero recibe s0 directamente en lugar de
  llamar a createSolution()
  return (s*, f*, evals)
end function
```

---



---

```
function ILS_ES(problem, maxevals)
  n ← problem.getSolutionSize()
  k ← problem.getSolutionDomainRange().second + 1
  num_iters ← 5;   evals_per_restart ← ⌊maxevals/num_iters⌋
  s0 ← problem.createSolution()
  (s*, f*, e) ← RUNES(problem, s0, evals_per_restart);   evals_total ← e
  for i ← 1 hasta num_iters − 1 do
    sm ← MUTATE(s*, k)                                ▷ Mismo operador que ILS
    (s', f', e) ← RUNES(problem, sm, evals_per_restart)
    evals_total += e
    if f' < f* then;   s* ← s';   f* ← f';
    end if
  end for
  return (s*, f*, evals_total)
end function
```

---

### 3.5. GRASP (variante voluntaria)

El GRASP es un procedimiento constructivo iterativo que combina una fase de construcción greedy aleatorizada con una fase de mejora mediante BL. La fase constructiva asigna las instancias una a una, guiándose tanto por criterios de infeasibility como de proximidad al centroide del cluster destino, e introduce aleatoriedad para diversificar las soluciones generadas.

En cada iteración, para cada instancia sin asignar se calcula el incremento de infeasibility que supondría asignarla a cada cluster (considerando solo las instancias ya asignadas). Los clusters que alcanzan el mínimo incremento forman la Lista de

Candidatos (LC). Esta lista se ordena de forma ascendente por la distancia de la instancia al centroide actual del cluster, y se restringe a los  $\lceil \alpha \cdot |LC| \rceil$  mejores candidatos, formando la Lista Restringida de Candidatos (LRC) con  $\alpha = 0,3$ . La instancia se asigna a un cluster elegido uniformemente al azar de la LRC. Los centroides se mantienen de forma incremental durante la construcción, sin recalcular desde cero en cada paso.

Se realizan 5 iteraciones completas (construcción + BL), con 20 000 evaluaciones por cada BL, devolviendo la mejor solución encontrada en todo el proceso.

---

---

```

function CONSTRUCTSOLUTION(problem, k,  $\alpha$ )
   $n \leftarrow problem.getSolutionSize()$ ;  $s \leftarrow [-1, \dots, -1]$ 
   $order \leftarrow [0, \dots, n-1]$  barajado aleatoriamente
  for  $c \leftarrow 0$  hasta  $k-1$  do
     $s[order[c]] \leftarrow c$ ; actualizar  $centroide_c$   $\triangleright$  Un elemento forzado por cluster
  end for
  for  $idx \leftarrow k$  hasta  $n-1$  do
     $i \leftarrow order[idx]$ 
    for  $c \leftarrow 0$  hasta  $k-1$  do
       $\Delta infeas_c \leftarrow$  restricciones violadas al asignar  $i$  a  $c$  (con instancias ya asignadas)
    end for
     $\mathcal{LC} \leftarrow \{c \mid \Delta infeas_c = \min_c \Delta infeas_c\}$ 
    Ordenar  $\mathcal{LC}$  ascendentemente por distancia de  $i$  al centroide de  $c$ 
     $\mathcal{LRC} \leftarrow$  primeros  $\max(1, \lceil \alpha \cdot |\mathcal{LC}| \rceil)$  elementos de  $\mathcal{LC}$ 
     $c^* \leftarrow$  elemento aleatorio uniforme de  $\mathcal{LRC}$ 
     $s[i] \leftarrow c^*$ ; actualizar  $centroide_{c^*}$  de forma incremental
  end for
  return  $s$ 
end function

```

---



---

---

```

function GRASP(problem, maxevals)
   $k \leftarrow problem.getSolutionDomainRange().second + 1$ ;  $\alpha \leftarrow 0,3$ 
   $num\_iters \leftarrow 5$ ;  $evals\_per\_restart \leftarrow \lfloor maxevals/num\_iters \rfloor$ 
   $s^* \leftarrow \emptyset$ ;  $f^* \leftarrow +\infty$ ;  $evals\_total \leftarrow 0$ 
  for  $i \leftarrow 1$  hasta  $num\_iters$  do
     $s_c \leftarrow CONSTRUCTSOLUTION(problem, k, \alpha)$   $\triangleright$  Genera solución inicial válida con criterio greedy aleatorizado
     $(s', f', e) \leftarrow LOCALSEARCH(problem, evals\_per\_restart, s_c)$ 
     $evals\_total += e$ 
    if  $f' < f^*$  then  $s^* \leftarrow s'$ ;  $f^* \leftarrow f'$ 
  end if
end for
  return  $(s^*, f^*, evals\_total)$ 
end function

```

---

### 3.6. ES-Fast (variante voluntaria)

El ES-Fast es una variante del ES que aprovecha el método `calculateInfeasibilityDelta()` de `ProblemPAR` para reducir el coste computacional de evaluar vecinos rechazados. La observación clave es que la función objetivo tiene dos componentes (desviación e infeasibility) y que calcular la desviación requiere recalculer los centroides, lo que es costoso. Para los vecinos que serán rechazados, el término de infeasibility suele ser el dominante en las fases iniciales de la búsqueda.

La estrategia consiste en calcular primero únicamente  $\Delta f_{\text{approx}} = \lambda \cdot \Delta \text{infeas}$ , con coste  $O(\text{restricciones de la instancia})$  frente al  $O(n^2)$  de la evaluación completa. Si con este delta aproximado el movimiento ya puede ser rechazado con certeza, se descarta sin evaluar la desviación. Solo si el movimiento es prometedor se computa el fitness completo. Esto permite explorar un mayor número de vecinos dentro del mismo presupuesto de evaluaciones, obteniendo en la práctica mejores soluciones que el ES estándar, especialmente en datasets con muchas restricciones.

---

```

function SIMULATEDANNEALINGFAST(problem, maxevals)
  Inicialización idéntica a SIMULATEDANNEALING:
     $s \leftarrow \text{problem.createSolution}()$ ; calcular  $T_0, \beta, M$ 
  Mantener current_infeas y current_dev por separado
  while evals < maxevals do
    ... (bucle externo e interno igual que ES)
     $i \leftarrow \sigma[\text{idx}]$ ;  $c' \leftarrow \text{cluster aleatorio} \neq s_i$ 
     $\Delta \text{infeas} \leftarrow \text{problem.calculateInfeasibilityDelta}(s, i, s_i, c')$ 
     $\Delta f_{\text{approx}} \leftarrow \lambda \cdot \Delta \text{infeas}$  ▷ Coste  $O(\text{restricciones de } i)$ 
    evals += 1; n_vecinos += 1
    if  $\Delta f_{\text{approx}} > 0$  and  $U(0, 1) > e^{-\Delta f_{\text{approx}}/T}$  then
      continue ▷ Rechazado sin evaluar desviación
    end if
     $s' \leftarrow s$  con  $s'_i \leftarrow c'$ ;  $f_{s'} \leftarrow \text{problem.fitness}(s')$  ▷ Solo si es prometedor
     $\Delta f \leftarrow f_{s'} - f_s$ 
    if  $\Delta f < 0$  or  $U(0, 1) \leq e^{-\Delta f/T}$  then
       $s \leftarrow s'$ ;  $f_s \leftarrow f_{s'}$ ; actualizar current_infeas, current_dev
      n_exitos += 1; si  $f_s < f^*$  entonces  $s^* \leftarrow s$ ;  $f^* \leftarrow f_s$ 
    end if
     $T \leftarrow T / (1 + \beta \cdot T)$ ; si n_exitos = 0 entonces break
  end while
  return ( $s^*, f^*, \text{evals}$ )
end function

```

---

## 4. Estructura del código

El proyecto sigue una arquitectura basada en la plantilla proporcionada en PRADO, con adaptaciones y extensiones específicas para el problema PAR y el análisis de resultados. El directorio principal `software/` se organiza de la siguiente manera:

- `common/`: Contiene las clases base y utilidades generales (`mh.h`, `problem.h`, `solution.h`, etc.). Estos archivos proporcionan la interfaz fundamental de la que heredan nuestros algoritmos y representaciones.
- `inc/` y `src/`: Directorios que separan, respectivamente, las cabeceras (`.h`) y las implementaciones (`.cpp`) de las clases desarrolladas para esta práctica. Incluyen la definición del problema (`problemPAR`) y la lógica de todos los algoritmos (`bmb`, `geneticalgorithm`, `grasp`, `greedy`, `localsearch`, `randomsearch`, `simulatedannealing`, `simulatedannealingfast`, `ils`, `ils.es`) implementando el método `optimize` para cada uno.
- `bin/`: Carpeta destino donde el compilador genera los archivos objeto (`.o`) y el ejecutable final (`par-solver`).
- `data/`: Directorio que almacena los conjuntos de datos y los archivos de restricciones a evaluar.
- **Archivos en la raíz:**
  - `main.cpp`: Punto de entrada del programa que gestiona los parámetros por consola y lanza la optimización.
  - `Makefile`: Archivo de configuración para automatizar la compilación.
  - `run_all_tests.sh`: Script en Bash para la ejecución en lote de toda la batería de pruebas.
  - `graf.py`: Script en Python para la generación automática de *boxplots* a partir de los resultados.
  - `wilcoxon.py`: Script en Python que realiza el test de Wilcoxon.
  - Archivos `.csv` y directorio `graficas/`: Salidas generadas tras la ejecución de los experimentos.

### 4.1. Manual de compilación

Para compilar el proyecto se ha incluido un `Makefile` configurado para el compilador `g++`. Se utiliza el estándar `C++17` y el flag de máxima optimización `-O3` para acelerar las ejecuciones.

Para generar el ejecutable, basta con abrir un terminal en el directorio raíz del proyecto y ejecutar el comando:

```
$ make
```

Esto compilará los archivos fuente y generará el ejecutable `par-solver` dentro de la carpeta `bin/`. Si se desea limpiar el entorno eliminando los archivos compilados, se puede ejecutar:

```
$ make clean
```



## 4.2. Manual de ejecución

El software ofrece flexibilidad para ejecutarse de forma individual (para pruebas concretas) o de forma automatizada (para extraer todos los resultados de la memoria).

### 1. Ejecución individual

El ejecutable principal admite hasta 4 parámetros por línea de comandos, siguiendo la sintaxis:

```
$ ./bin/par_solver <dataset_base_path> <k_clusters> <algorithm> [seed]
```

- `<dataset_base_path>`: Ruta base del archivo de datos (ej. `data/glass_set`). El programa buscará automáticamente su archivo de restricciones asociado.
- `<k_clusters>`: Número de clusters a formar (típicamente 15 o 30).
- `<algorithm>`: Algoritmo a ejecutar (`random`, `greedy`, `local`, `am-best`, `es`, `es-fast`, `bmb`, `ils`, `ils-es`, `grasp`, o `all` para ejecutarlos todos secuencialmente).
- `[seed]` (*Opcional*): Semilla para el generador de números aleatorios. Si no se especifica, el programa realiza 10 ejecuciones independientes usando un conjunto de semillas por defecto.

*Ejemplo de ejecución:* Ejecutar todos los algoritmos sobre el dataset *zoo* con  $k = 15$  y semilla 42:

```
$ ./bin/par_solver data/zoo_set 15 all 42
```

### 2. Ejecución automatizada (Batería de pruebas)

Para replicar los resultados mostrados en esta memoria, se ha desarrollado el script de bash `run_all_tests.sh`. Este script lanza internamente el programa para los datasets Bupa, Glass y Zoo (con  $k = 15$  y  $k = 30$ ) empleando las 10 semillas.

```
$ ./run_all_tests.sh
```

Al finalizar, el programa genera automáticamente los archivos `det_results.csv` (resultados detallados) y `avg_results.csv` (medias de tiempo, fitness y evaluaciones).

### 3. Generación de Gráficas

Una vez generados los archivos CSV, se pueden obtener los *boxplots* que comparen la estabilidad de los algoritmos ejecutando el script de Python:

```
$ python3 graf.py
```

Este comando leerá el archivo de resultados detallados y exportará las imágenes PNG dentro de la carpeta `graficas/`.

## 5. Experimentación y Análisis de Resultados

### 5.1. Descripción de los casos de prueba

Para evaluar el rendimiento y la robustez de los algoritmos implementados, se ha seleccionado un banco de pruebas compuesto por tres conjuntos de datos (*datasets*) de características variadas. Para cada uno de estos conjuntos, se ha abordado el problema de particionamiento considerando dos valores distintos para el número de clusters a formar:  $k = 15$  y  $k = 30$ .

Las características principales de los conjuntos de datos, incluyendo el número total de instancias ( $n$ ) y el número de atributos numéricos continuos que definen cada instancia ( $d$ ), se resumen en la siguiente tabla:

| Dataset (.dat) | Instancias ( $n$ ) | Atributos ( $d$ ) |
|----------------|--------------------|-------------------|
| bupa_set       | 345                | 5                 |
| glass_set      | 214                | 9                 |
| zoo_set        | 101                | 16                |

Cuadro 1: Características de los datasets empleados.

Junto a cada conjunto de datos y para cada valor de  $k$ , se proporciona un archivo de restricciones asociado (`_const_15.dat` y `_const_30.dat`), que define las condiciones estructurales *Must-Link* y *Cannot-Link* que los algoritmos deben intentar satisfacer.

### 5.2. Parámetros de ejecución

Al igual que en prácticas anteriores, dado el carácter estocástico de los algoritmos evaluados, se define un entorno de pruebas riguroso que permita extraer conclusiones con significancia estadística. La configuración del marco experimental es la siguiente:

- **Criterio de parada:** Todos los algoritmos estocásticos implementados (ES, BMB, ILS, ILS-ES y las variantes voluntarias GRASP y ES-Fast) utilizan como criterio de parada un límite máximo estricto de 100 000 evaluaciones de la función objetivo. En el caso del ES y el ES-Fast existe además una condición de parada anticipada: si el número de éxitos en un paso de enfriamiento es cero, el algoritmo finaliza antes de alcanzar dicho límite.
- **Parámetros del Enfriamiento Simulado (ES y ES-Fast):** La temperatura inicial se calcula dinámicamente a partir del coste de la solución de partida:  $T_0 = \mu \cdot f(s_0) / (-\ln \varphi)$ , con  $\mu = 0,2$  y  $\varphi = 0,3$ . La temperatura final se fija en  $T_f = 10^{-3}$ . El número de enfriamientos es  $M = \lfloor 100\,000 / \max\_vecinos \rfloor$ , con  $\max\_vecinos = 10n$  (siendo  $n$  el tamaño de la solución). El bucle interno finaliza al generar  $\max\_vecinos = 10n$  vecinos o al aceptar  $\max\_ exitos = n$ . El coeficiente de enfriamiento  $\beta$  del esquema de Cauchy modificado se calcula automáticamente a partir de  $T_0$ ,  $T_f$  y  $M$ .

- **Parámetros de BMB:** Se realizan 5 reinicios independientes, cada uno con un presupuesto de  $\lfloor 100\,000/5 \rfloor = 20\,000$  evaluaciones para la BL. No existen parámetros adicionales.
- **Parámetros de ILS:** Se realizan 5 iteraciones (1 inicial + 4 sobre soluciones mutadas), con 20 000 evaluaciones de BL por iteración. El operador de mutación modifica el 20 % de los genes de la mejor solución encontrada, asignando a cada gen seleccionado un cluster distinto del original de forma uniforme aleatoria.
- **Parámetros de ILS-ES:** Estructura idéntica al ILS (5 iteraciones, mismo operador de mutación del 20 %), pero el optimizador interno es el ES con los mismos parámetros descritos anteriormente y un presupuesto de 20 000 evaluaciones por llamada.
- **Parámetros de GRASP:** Se realizan 5 iteraciones de construcción + BL, con 20 000 evaluaciones de BL por iteración. El parámetro  $\alpha = 0,3$  controla el tamaño de la Lista Restringida de Candidatos ( $LRC = \lceil 0,3 \cdot |LC| \rceil$  mejores candidatos de la LC).
- **Inicialización del generador aleatorio:** Para mitigar el sesgo introducido por la aleatoriedad, se han ejecutado 10 iteraciones independientes para cada combinación de algoritmo, *dataset* y valor de  $k$ . Las ejecuciones se controlan mediante el conjunto de semillas consecutivas  $\{1000, 1001, \dots, 1009\}$ , empleadas para inicializar el generador de números aleatorios mediante `Random::seed()`. La ejecución  $i$ -ésima de cada algoritmo utiliza la misma semilla que la de todos los demás, lo que permite realizar comparaciones emparejadas bajo idénticas condiciones iniciales, garantizando tanto la reproducibilidad de los experimentos como la validez de los contrastes estadísticos posteriores.
- **Registro de resultados:** Durante cada ejecución, el sistema registra de manera automatizada el *fitness* alcanzado, la desviación intra-clúster, la *infeasibility* final, el número real de evaluaciones consumidas y el tiempo de cómputo en segundos.

### 5.3. Resultados obtenidos

En esta sección se presentan los resultados obtenidos en los experimentos realizados. Las tablas siguientes muestran los valores medios obtenidos a partir de dichas 10 ejecuciones. En concreto, se reportan las medias del valor de la función objetivo *fitness*, la desviación intra-cluster, la *infeasibility* (número de restricciones violadas), el número de evaluaciones de la función objetivo realizadas y el tiempo de ejecución en segundos.

Además de la presentación de los valores medios, se incluirán más tarde<sup>2</sup> diagramas *boxplots* que permiten visualizar la distribución de los resultados obtenidos en las 10 ejecuciones realizadas para cada algoritmo.

<sup>2</sup>en la sección 5.4.5: Análisis de la variabilidad de los resultados

| Algoritmo | Fitness | Distancia | Incumplimiento | Evaluaciones | Tiempo (s) |
|-----------|---------|-----------|----------------|--------------|------------|
| Random    | 0.34    | 0.25      | 493.5          | 100000       | 17.51      |
| Greedy    | 0.24    | 0.23      | 22.7           | 1            | 0.02       |
| Local     | 0.18    | 0.15      | 168.2          | 85442.6      | 14.04      |
| AM-Best   | 0.19    | 0.16      | 158.7          | 100000       | 16.52      |
| ES        | 0.24    | 0.18      | 321.2          | 100000       | 16.68      |
| ES-Fast   | 0.24    | 0.21      | 182            | 100000       | 0.21       |
| BMB       | 0.19    | 0.15      | 187.1          | 100000       | 16.85      |
| ILS-BL    | 0.18    | 0.15      | 168.1          | 100000       | 16.58      |
| ILS-ES    | 0.23    | 0.18      | 291.3          | 100000       | 16.58      |
| GRASP     | 0.19    | 0.16      | 169.5          | 100000       | 16.53      |

Cuadro 2: Resultados medios para el dataset *Bupa* con  $k = 15$ .

| Algoritmo | Fitness | Distancia | Incumplimiento | Evaluaciones | Tiempo (s) |
|-----------|---------|-----------|----------------|--------------|------------|
| Random    | 0.28    | 0.23      | 551.1          | 100000       | 29.86      |
| Greedy    | 0.22    | 0.22      | 9              | 1            | 0.04       |
| Local     | 0.13    | 0.1       | 346.7          | 99781.7      | 29.23      |
| AM-Best   | 0.14    | 0.11      | 328.2          | 100000       | 29.26      |
| ES        | 0.19    | 0.13      | 644.5          | 100000       | 29.07      |
| ES-Fast   | 0.2     | 0.16      | 395.6          | 100000       | 0.34       |
| BMB       | 0.14    | 0.1       | 391.7          | 100000       | 29.6       |
| ILS-BL    | 0.13    | 0.1       | 355.5          | 100000       | 29.38      |
| ILS-ES    | 0.19    | 0.14      | 588.3          | 100000       | 28.92      |
| GRASP     | 0.14    | 0.11      | 370.6          | 100000       | 29.45      |

Cuadro 3: Resultados medios para el dataset *Bupa* con  $k = 30$ .

| Algoritmo | Fitness | Distancia | Incumplimiento | Evaluaciones | Tiempo (s) |
|-----------|---------|-----------|----------------|--------------|------------|
| Random    | 0.52    | 0.4       | 177.7          | 100000       | 6.51       |
| Greedy    | 0.33    | 0.33      | 0              | 1            | 0.01       |
| Local     | 0.22    | 0.19      | 47.2           | 36397        | 2.15       |
| AM-Best   | 0.22    | 0.19      | 43.4           | 100000       | 5.8        |
| ES        | 0.25    | 0.2       | 72.4           | 100000       | 5.74       |
| ES-Fast   | 0.26    | 0.24      | 27.9           | 100000       | 0.14       |
| BMB       | 0.22    | 0.19      | 50.9           | 100000       | 5.84       |
| ILS-BL    | 0.21    | 0.19      | 43.3           | 99722.1      | 5.78       |
| ILS-ES    | 0.26    | 0.21      | 72.2           | 100000       | 5.78       |
| GRASP     | 0.22    | 0.19      | 45.7           | 100000       | 5.81       |

Cuadro 4: Resultados medios para el dataset *Glass* con  $k = 15$ .

| Algoritmo | Fitness | Distancia | Incumplimiento | Evaluaciones | Tiempo (s) |
|-----------|---------|-----------|----------------|--------------|------------|
| Random    | 0.43    | 0.36      | 213.5          | 100000       | 11.23      |
| Greedy    | 0.28    | 0.28      | 0              | 1            | 0.01       |
| Local     | 0.16    | 0.13      | 103.6          | 76518.1      | 7.97       |
| AM-Best   | 0.15    | 0.12      | 92.8           | 100000       | 10.37      |
| ES        | 0.2     | 0.15      | 179.3          | 100000       | 10.42      |
| ES-Fast   | 0.21    | 0.18      | 103            | 100000       | 0.23       |
| BMB       | 0.17    | 0.13      | 112.3          | 100000       | 10.61      |
| ILS-BL    | 0.16    | 0.13      | 106            | 94000.3      | 9.89       |
| ILS-ES    | 0.21    | 0.15      | 174.6          | 100000       | 10.41      |
| GRASP     | 0.16    | 0.13      | 109.6          | 100000       | 10.47      |

Cuadro 5: Resultados medios para el dataset *Glass* con  $k = 30$ .

| Algoritmo | Fitness | Distancia | Incumplimiento | Evaluaciones | Tiempo (s)            |
|-----------|---------|-----------|----------------|--------------|-----------------------|
| Random    | 1.51    | 1.3       | 37.4           | 100000       | 1.33                  |
| Greedy    | 0.89    | 0.89      | 0              | 1            | $1,29 \times 10^{-3}$ |
| Local     | 0.53    | 0.5       | 6.3            | 13951.9      | 0.12                  |
| AM-Best   | 0.51    | 0.48      | 4.5            | 100000       | 0.84                  |
| ES        | 0.45    | 0.43      | 5              | 100000       | 0.83                  |
| ES-Fast   | 0.5     | 0.5       | 0.5            | 100000       | 0.14                  |
| BMB       | 0.49    | 0.44      | 9              | 63847.6      | 0.55                  |
| ILS-BL    | 0.49    | 0.45      | 7.1            | 53388.6      | 0.46                  |
| ILS-ES    | 0.47    | 0.42      | 8.4            | 100000       | 0.84                  |
| GRASP     | 0.48    | 0.46      | 5.1            | 57408.2      | 0.48                  |

Cuadro 6: Resultados medios para el dataset *Zoo* con  $k = 15$ .

| Algoritmo | Fitness | Distancia | Incumplimiento | Evaluaciones | Tiempo (s)            |
|-----------|---------|-----------|----------------|--------------|-----------------------|
| Random    | 1.06    | 0.93      | 48.9           | 100000       | 1.81                  |
| Greedy    | 0.6     | 0.6       | 0              | 1            | $2,15 \times 10^{-3}$ |
| Local     | 0.29    | 0.24      | 17.5           | 23325.4      | 0.24                  |
| AM-Best   | 0.29    | 0.25      | 18.9           | 100000       | 1.1                   |
| ES        | 0.27    | 0.22      | 19.3           | 100000       | 1.04                  |
| ES-Fast   | 0.3     | 0.29      | 4.6            | 99688.9      | 0.2                   |
| BMB       | 0.28    | 0.23      | 20.2           | 91723.2      | 0.96                  |
| ILS-BL    | 0.28    | 0.24      | 14.7           | 64400.3      | 0.69                  |
| ILS-ES    | 0.29    | 0.24      | 19.1           | 100000       | 1.01                  |
| GRASP     | 0.27    | 0.22      | 18.3           | 90806.1      | 0.98                  |

Cuadro 7: Resultados medios para el dataset *Zoo* con  $k = 30$ .

#### 5.4. Análisis de resultados

Una vez presentados los resultados promedios de las ejecuciones, procedemos a realizar un análisis exhaustivo del comportamiento de los diferentes algoritmos basados

en trayectorias implementados. En esta sección se evaluará cómo la introducción de mecanismos de escape de óptimos locales (Enfriamiento Simulado y Búsqueda Local Iterada) y la generación inteligente de soluciones iniciales (GRASP) impactan en la calidad de las soluciones, comparándolos tanto entre sí como con el mejor exponente poblacional de la práctica anterior (AM-Best).

#### 5.4.1. Comparación general entre algoritmos

A partir de los resultados observados, se evidencia una clara jerarquía en el rendimiento fuertemente condicionada por la complejidad del espacio de búsqueda de cada dataset. La *Búsqueda Aleatoria* (RandomSearch) establece el límite inferior de calidad en todos los casos, obteniendo sistemáticamente los peores valores de *fitness* (por ejemplo, 1,51 en *Zoo* con  $k = 15$ , como puede verse en el Cuadro 6).

La *Búsqueda Local* pura (LocalSearch) demuestra ser un explotador sumamente eficiente en paisajes sencillos. De hecho, en *Bupa* con  $k = 30$ , logra el mejor *fitness* promedio absoluto de la tabla (0,13, en Cuadro 3), empatando con ILS-BL y superando incluso a AM-Best (0,14). Sin embargo, su debilidad estructural (el estancamiento en óptimos locales) queda al descubierto en problemas más complejos topológicamente como *Zoo*. Es en estos escenarios donde destacan las metaheurísticas de trayectoria avanzadas.

El ES destaca como el algoritmo de trayectoria simple más potente en datasets compactos como *Zoo*: obtiene el mejor *fitness* absoluto en ambas configuraciones (0,45 con  $k = 15$  y 0,27 con  $k = 30$ , Cuadros 6 y 7, resp.), por encima de todos los demás algoritmos incluido AM-Best (0,51 y 0,29 resp.). Esta ventaja se explica por su capacidad de aceptar soluciones temporalmente peores, que le permite atravesar barreras de infactibilidad para encontrar valles de *fitness* inaccesibles para la BL pura.

El ILS-BL se revela como el algoritmo de trayectoria múltiple más robusto del conjunto: iguala o supera a la BL pura en todos los casos excepto *Bupa*  $k = 30$  (donde ambos empatan en 0,13), y supera a AM-Best en la mayoría de escenarios. En *Zoo*  $k = 30$  (Cuadro 7) obtiene 0,279 frente al 0,293 del memético, confirmando que la intensificación reiterada sobre una única solución perturbada resulta altamente competitiva frente a la evolución de una población entera.

Un resultado llamativo es el ILS-ES: pese a combinar los dos mecanismos más potentes, su rendimiento no supera al ES independiente en ningún caso y en varios resulta notablemente peor (p.ej., 0,47 vs 0,45 en *Zoo*  $k = 15$ , Cuadro 6). La causa probable es el fraccionamiento del presupuesto: dividir 100 000 evaluaciones en  $5 \times 20\,000$  impide que cada instancia del ES alcance la temperatura suficientemente baja para converger correctamente, mientras que el ES completo con 100 000 evaluaciones sí lo logra. Esto sugiere que el ES es sensiblemente más sensible al presupuesto total que la BL, cuyos reinicios en ILS-BL funcionan mejor con presupuestos pequeños.

#### 5.4.2. Impacto de las restricciones en la función objetivo

El análisis detallado de las componentes de la función objetivo revela cómo cada mecanismo gestiona el delicado compromiso entre la compacidad geométrica (Distancia) y la factibilidad relacional (Incumplimiento).

El algoritmo *Greedy* sigue destacando por generar soluciones iniciales con un incumplimiento nulo o casi nulo (como los 0 incumplimientos en todos los escenarios de *Glass* y *Zoo*), a costa de distancias inter-clúster mediocres que resultan en un *fitness* global peor que los algoritmos basados en BL. Por el contrario, los algoritmos que incorporan BL tienden a sacrificar la factibilidad para ganar en distancia.

El comportamiento del ES frente a este compromiso es especialmente revelador. En datasets con un espacio de restricciones denso como *Bupa*, la aceptación de peores soluciones dispara el incumplimiento (644,5 con  $k = 30$ , casi el doble de los 346,7 de la BL pura, Cuadro 3), sin que el beneficio en distancia compense la penalización por  $\lambda$ : el *fitness* resultante de 0,192 es 47 % peor que el de la BL (0,128). En cambio, en *Zoo*, el mismo mecanismo exploratorio le permite cruzar barreras de infactibilidad temporal para alcanzar mínimos globales inaccesibles, con apenas 5,0 incumplimientos de media en  $k = 15$ , como se aprecia en Cuadro 6.

El patrón del GRASP resulta también interesante: su fase constructiva greedy aleatorizada produce soluciones iniciales con incumplimientos notablemente menores que los de BMB (p.ej., 45,7 vs 50,9 en *Glass*  $k = 15$ , Cuadro 4; 18,3 vs 20,2 en *Zoo*  $k = 30$ , Cuadro 7), lo que sugiere que arrancar desde una solución más factible beneficia a la BL posterior, que puede focalizarse en mejorar la desviación en lugar de reparar restricciones.

#### 5.4.3. Esfuerzo computacional y convergencia

La BL converge prematuramente en los datasets pequeños: apenas consume 13 952 evaluaciones en *Zoo*  $k = 15$  y 36 397 en *Glass*  $k = 15$ , mientras que en *Bupa* sí se aproxima al límite. Esta convergencia temprana es la que justifica los mecanismos de reinicio: BMB, ILS y GRASP explotan el presupuesto completo de 100 000 evaluaciones en *Bupa* y *Glass*, y aún en *Zoo* consumen el doble o el triple que la BL antes de detenerse. La capacidad de seguir explorando tras el primer estancamiento explica su superioridad en problemas de alta dificultad.

#### 5.4.4. Comparación entre conjuntos de datos y dimensionalidad

Aumentar de  $k = 15$  a  $k = 30$  facilita reducir la desviación intra-clúster pero eleva el incumplimiento de restricciones ML y CL, haciendo el problema más difícil para todos los algoritmos. Este efecto es especialmente pronunciado en *Bupa*, donde el espacio de restricciones es más denso, y penaliza más al ES que a los algoritmos de trayectoria múltiple.

En cuanto a BMB y GRASP, en  $k = 15$  sus resultados son prácticamente equivalentes en los tres datasets. La diferencia emerge con  $k = 30$ : GRASP alcanza 0,269 frente a 0,280 de BMB en *Zoo*, y 0,160 frente a 0,165 en *Glass*. La construcción greedy añade valor precisamente cuando el espacio es mayor y más restringido, esto es, cuando la solución de partida importa más.

#### 5.4.5. Análisis de la variabilidad de los resultados

Como se ha mencionado previamente, con el objetivo de evaluar la estabilidad de los algoritmos, cada configuración se ejecutó 10 veces sobre cada dataset para distintos

valores de  $k$ . A partir de estas ejecuciones se construyeron diagramas de caja (*box-plots*), que permiten analizar la distribución del *fitness*, su dispersión y la presencia de posibles valores atípicos (*outliers*).

Los gráficos se organizan en cuatro tipos de comparativa: visión global, mecanismos de escape (trayectoria simple), multi-arranque aleatorio vs guiado, y trayectorias avanzadas frente al mejor memético de la práctica anterior.

**Dataset *bupa\_set*** La Figura 1 muestra la comparativa global para  $k = 15$  y  $k = 30$ . En ambos valores se aprecia inmediatamente la gran separación entre dos grupos: por un lado, *Random*, *Greedy*, ES, ES-Fast e ILS-ES, que se mantienen en franjas altas de *fitness*; y por otro, *Local*, ILS, BMB y GRASP, que se sitúan en una franja notablemente inferior. Este dataset penaliza especialmente los mecanismos de exploración no acotada: ES presenta en  $k = 15$  un *outlier* inferior aislado ( $\sim 0,225$ ), lo que indica que en ocasiones encuentra una solución excelente, pero la mediana sigue en  $\sim 0,236$ , idéntica a Greedy. ES-Fast queda sistemáticamente por encima de ES, confirmando el coste en calidad de la aproximación incremental.

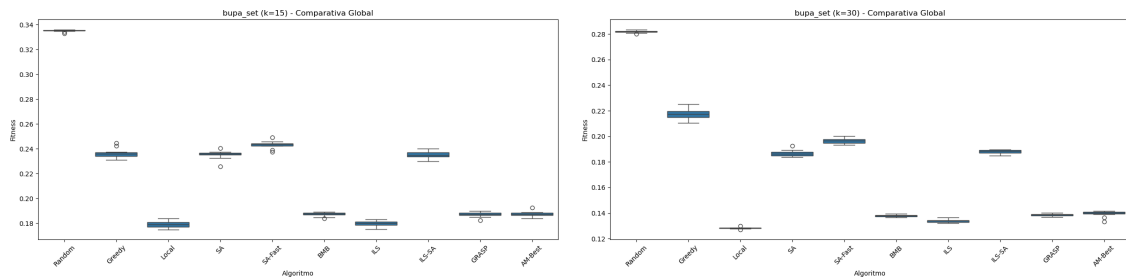


Figura 1: Comparativa global para el dataset *Bupa* con  $k = 15$  (izq.) y  $k = 30$  (dcha.)

La Figura 2 detalla el comportamiento de los mecanismos de escape de óptimos locales. Para  $k = 15$ , ILS y *Local* son prácticamente indistinguibles en mediana ( $\sim 0,180$ ) y variabilidad (cajas estrechas), lo que indica que en este escenario la perturbación del ILS no aporta mejora sobre arrancar desde cero: el óptimo local de la BL ya es de alta calidad. ES muestra mayor dispersión y peor mediana, y ES-Fast queda claramente rezagado ( $\sim 0,244$ ) con una caja más ancha. Para  $k = 30$ , la separación se amplía: ILS ( $\sim 0,133$ ) supera claramente a ES ( $\sim 0,187$ ) e ILS-ES ( $\sim 0,188$ ), confirmando que con mayor espacio de búsqueda la acumulación de múltiples reinicios perturbados sí marca diferencia respecto al esquema continuo del ES.



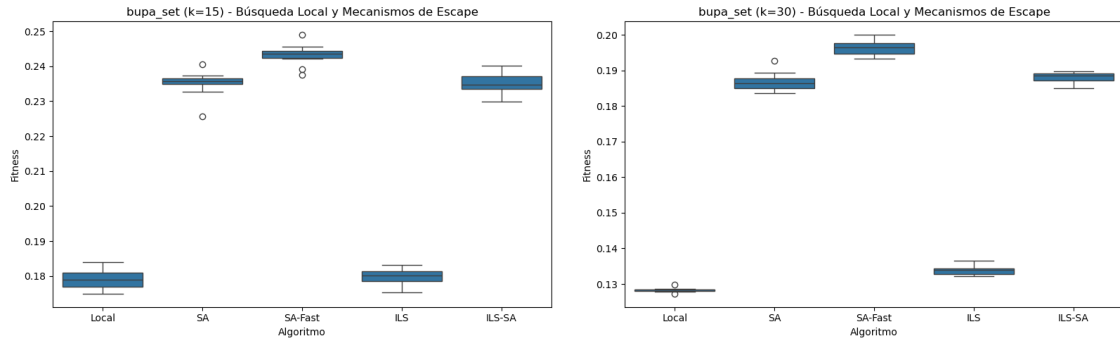


Figura 2: Mecanismos de escape para el dataset *Bupa* con  $k = 15$  (izq.) y  $k = 30$  (dcha.)

La Figura 3 compara las trayectorias avanzadas con AM-Best. En  $k = 15$ , *Local* e ILS son claramente los mejores ( $\sim 0,179$ – $0,180$ ), mientras que GRASP y AM-Best se agrupan en torno a  $\sim 0,187$ : la fase constructiva del GRASP no aporta ventaja en este dataset, donde los multi-arranques aleatorios de BMB son igualmente efectivos. En  $k = 30$ , el resultado es llamativo: *Local* sigue siendo el mejor ( $\sim 0,128$ , caja extremadamente estrecha), ILS se aproxima ( $\sim 0,134$ ), GRASP queda en  $\sim 0,139$  y AM-Best es el peor del grupo ( $\sim 0,140$ ) con variabilidad claramente mayor. En *Bupa*, la explotación pura y la perturbación reiterada superan tanto a la construcción inteligente como a la evolución poblacional.

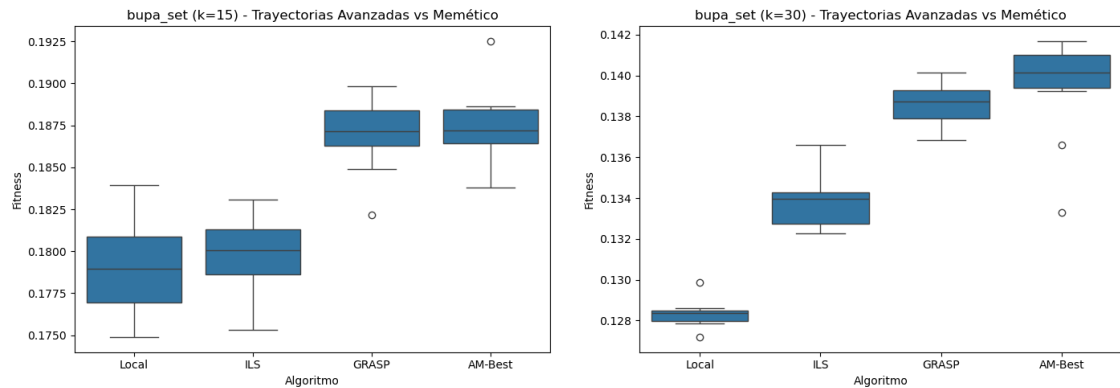


Figura 3: Trayectorias avanzadas vs AM-Best para el dataset *Bupa* con  $k = 15$  (izq.) y  $k = 30$  (dcha.)

**Dataset *glass\_set*** La Figura 4 muestra la comparativa global. Este dataset presenta el patrón más interesante de todos: ILS emerge como el mejor algoritmo de trayectoria en  $k = 15$  ( $\sim 0,214$ ), superando incluso a *Local* ( $\sim 0,222$ ) y a AM-Best ( $\sim 0,222$ ), con una caja notablemente más estrecha que el resto. ES y ES-Fast quedan en las posiciones más desfavorables del grupo de trayectorias ( $\sim 0,250$  y  $\sim 0,265$  respectivamente), lo que evidencia que en este dataset el espacio de búsqueda tiene una estructura que penaliza la exploración probabilística descontrolada.

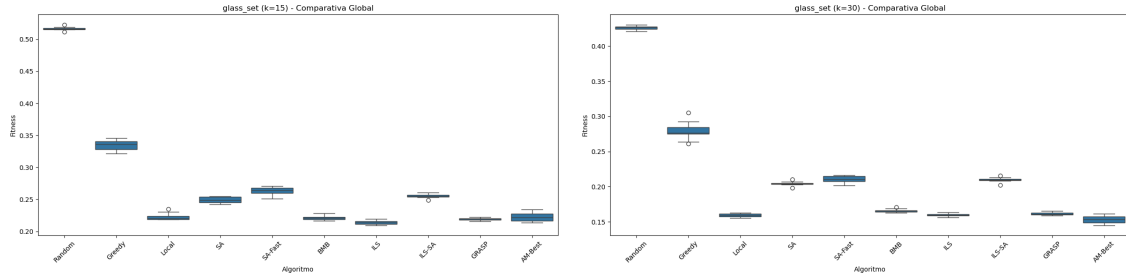


Figura 4: Comparativa global para el dataset *Glass* con  $k = 15$  (izq.) y  $k = 30$  (dcha.)

La Figura 5 confirma con mayor detalle esta jerarquía. En  $k = 15$ , ILS ( $\sim 0,214$ ) domina con claridad: su caja es la más estrecha y su mediana la más baja de todos los algoritmos mostrados, incluido *Local*. ES-Fast obtiene la peor mediana ( $\sim 0,265$ ) y la mayor variabilidad, mientras que ILS-ES ( $\sim 0,256$ ) queda también por detrás de ES ( $\sim 0,250$ ). En  $k = 30$ , ILS y *Local* empatan casi exactamente en mediana ( $\sim 0,160$ ), con cajas igualmente compactas, mientras que ES ( $\sim 0,204$ ), ES-Fast ( $\sim 0,213$ ) e ILS-ES ( $\sim 0,210$ ) quedan en una franja muy superior. La conclusión es inequívoca: en *Glass*, los mecanismos de escape del ES no compensan frente a los múltiples reinicios del ILS.

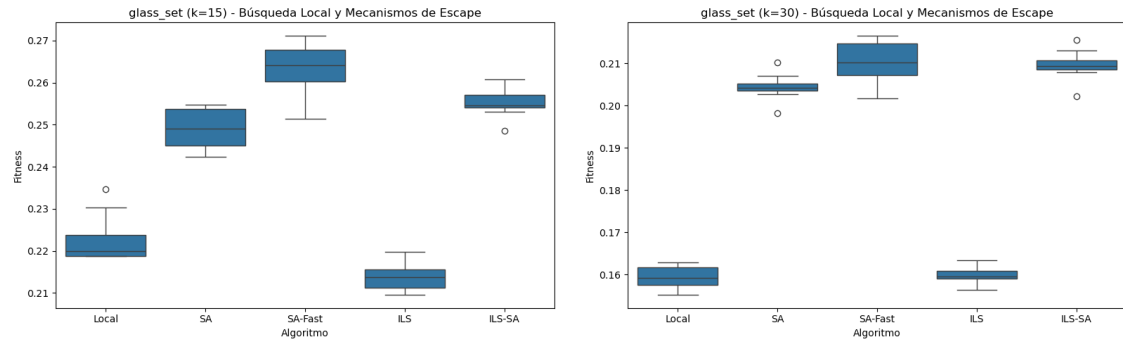


Figura 5: Mecanismos de escape para el dataset *Glass* con  $k = 15$  (izq.) y  $k = 30$  (dcha.)

La Figura 6 muestra la comparativa BMB vs GRASP. En  $k = 15$ , GRASP obtiene una mediana ligeramente inferior ( $\sim 0,220$  vs  $\sim 0,221$ ) y un rango intercuartílico más estrecho, con un mínimo absoluto menor ( $\sim 0,216$  vs  $\sim 0,217$ ). La diferencia es pequeña pero consistente. En  $k = 30$ , la ventaja del GRASP se amplía de forma apreciable: su caja ( $0,161$ – $0,163$ ) queda claramente por debajo de la de BMB ( $0,164$ – $0,166$ ), y su mínimo absoluto ( $\sim 0,160$ ) supera al de BMB ( $\sim 0,163$ ). Esto confirma que, conforme el espacio de clústeres crece, la fase constructiva greedy del GRASP (que minimiza el incumplimiento durante la propia construcción) proporciona puntos de partida cualitativamente mejores que los aleatorios de BMB.

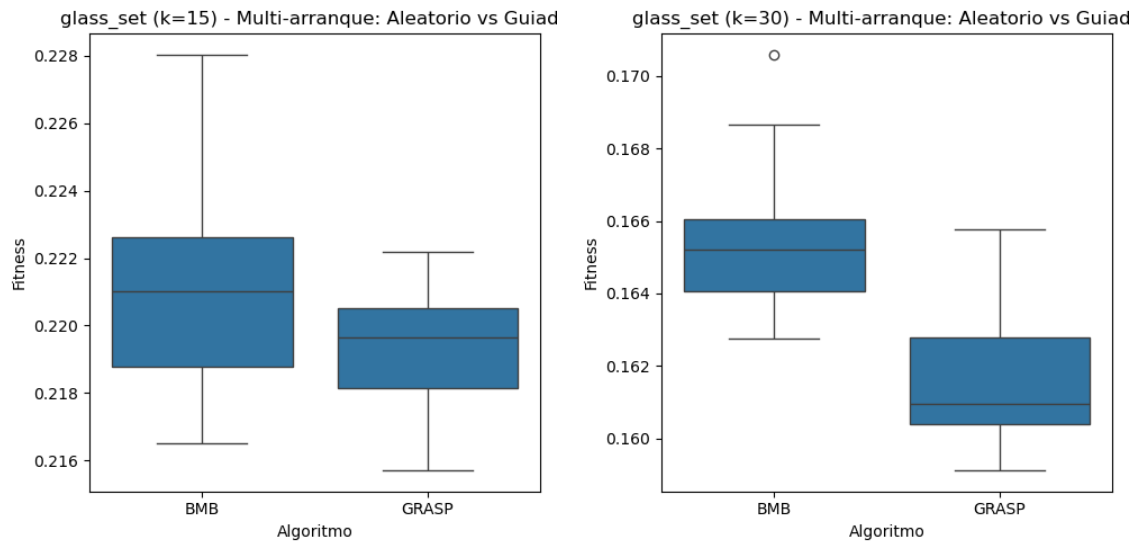


Figura 6: Multi-arranque: BMB vs GRASP para el dataset *Glass* con  $k = 15$  (izq.) y  $k = 30$  (dcha.)

Finalmente, la Figura 7 pone en evidencia uno de los resultados más llamativos de este estudio. En  $k = 15$ , ILS ( $\sim 0,214$ ) supera a todos: *Local*, GRASP y AM-Best se agrupan entre 0,219 y 0,222, mientras que AM-Best muestra la mayor variabilidad del grupo (caja más amplia y *outlier* superior en  $\sim 0,235$ ). En  $k = 30$ , AM-Best presenta una distribución bimodal característica: su caja se extiende entre  $\sim 0,149$  y  $\sim 0,158$ , con una mediana de  $\sim 0,153$  que sería la mejor del conjunto, pero con una variabilidad muy superior a ILS ( $\sim 0,160$ , caja estrecha) y *Local* ( $\sim 0,159$ , caja estrecha). En términos prácticos, AM-Best puede producir soluciones excelentes o mediocres dependiendo de la semilla, mientras que ILS y *Local* son predeciblemente buenos.

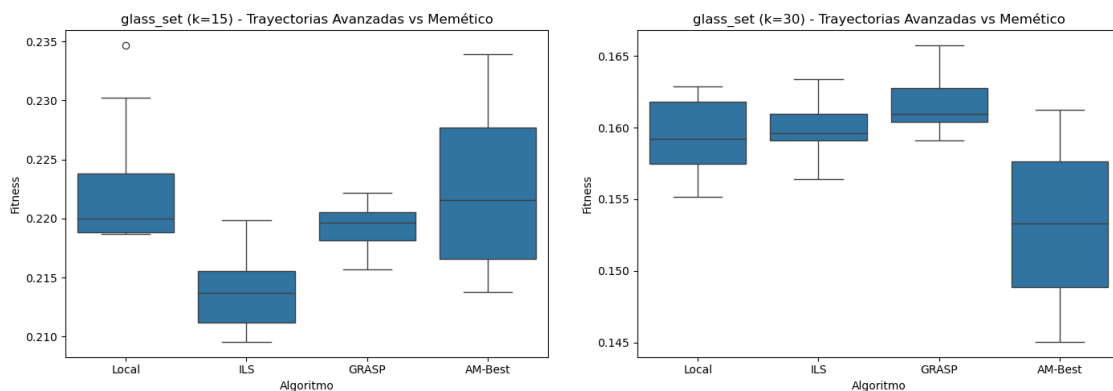


Figura 7: Trayectorias avanzadas vs AM-Best para el dataset *Glass* con  $k = 15$  (izq.) y  $k = 30$  (dcha.)

**Dataset *zoo.set*** La Figura 8 muestra la comparativa global para  $k = 15$  y  $k = 30$ . *Zoo* es el dataset con mayor dispersión relativa entre algoritmos y, a la vez,

el que presenta el patrón más distinto al observado en *Bupa* y *Glass*. En  $k = 15$ , *Local* exhibe la mayor variabilidad de todo el estudio (rango 0,455–0,610, con un *outlier* inferior aislado en  $\sim 0,455$ ), lo que refleja que la calidad del óptimo local depende fuertemente del punto de partida en este dataset. ES, por el contrario, aparece como el algoritmo más compacto y de menor mediana ( $\sim 0,450$ ), invirtiendo completamente la jerarquía observada en *Bupa* y *Glass*. En  $k = 30$ , el panorama se clarifica: ES y los algoritmos de trayectoria múltiple convergen en una banda estrecha (0,270–0,300), mientras que ES-Fast se desmarca negativamente con una variabilidad muy elevada.

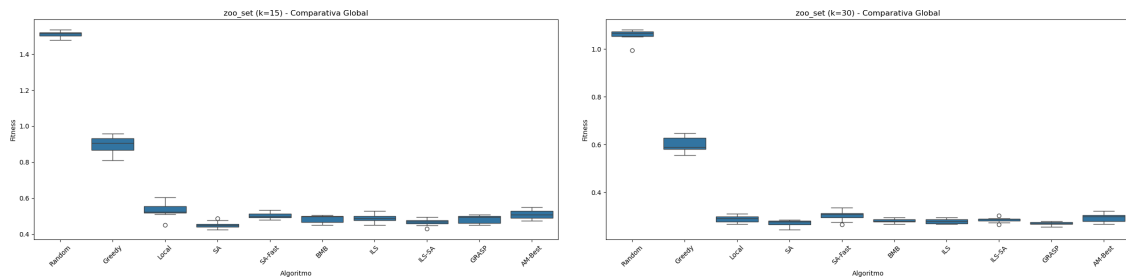


Figura 8: Comparativa global para el dataset *Zoo* con  $k = 15$  (izq.) y  $k = 30$  (dcha.)

La Figura 9 es la más reveladora de todo el análisis de *Zoo*. En  $k = 15$ , ES obtiene la mejor mediana ( $\sim 0,450$ ) y la caja más estrecha de todos los algoritmos de escape, con el rango intercuartílico más reducido (0,430–0,475). Su mecanismo de aceptación probabilística le permite atravesar barreras de infactibilidad para alcanzar valles de *fitness* inaccesibles para la BL pura, que queda con mediana  $\sim 0,525$ . Destaca también ILS-SA: aunque en los datasets anteriores nunca superaba a ES, aquí obtiene la segunda mejor mediana ( $\sim 0,468$ ) con baja dispersión y un *outlier* inferior muy prometedor ( $\sim 0,430$ ). La combinación de perturbación y ES parece encajar mejor con la topología del espacio de *Zoo*.

En  $k = 30$ , sigue siendo el mejor algoritmo de escape ( $\sim 0,278$ , caja compacta con bigote inferior hasta  $\sim 0,250$ ), mientras que ES-Fast sufre una degradación drástica: su caja abarca 0,295–0,310 con bigotes hasta 0,335 y un *outlier* inferior aislado en  $\sim 0,265$ . Esta alta varianza de ES-Fast en *Zoo* evidencia que el sesgo de la evaluación aproximada es especialmente perjudicial en este dataset, donde el balance entre desviación e infeasibility es más delicado.

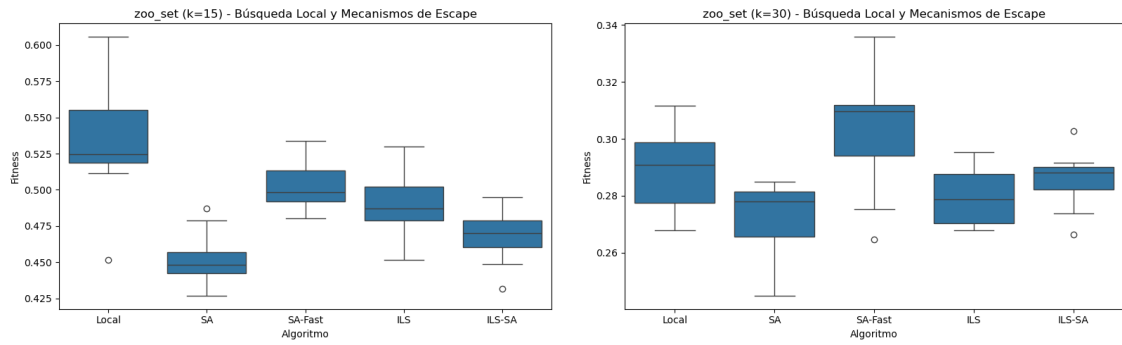


Figura 9: Mecanismos de escape para el dataset *Zoo* con  $k = 15$  (izq.) y  $k = 30$  (dcha.)

La Figura 10 muestra la comparativa BMB vs GRASP. En  $k = 15$ , ambos presentan cajas amplias y prácticamente solapadas ( $\sim 0,467$ – $0,500$  para ambos), siendo BMB ligeramente más compacto en el cuartil superior y GRASP con un mínimo absoluto algo menor ( $\sim 0,455$  vs  $\sim 0,455$ ). La diferencia es mínima. En  $k = 30$ , en cambio, la ventaja del GRASP es clara e inequívoca: su caja ( $0,268$ – $0,277$ ) queda enteramente por debajo de la de BMB ( $0,277$ – $0,286$ ), con mediana  $\sim 0,271$  frente a  $\sim 0,280$  de BMB, y un mínimo absoluto de  $\sim 0,256$  frente a  $\sim 0,267$ . Con más clusters y mayor complejidad relacional, la construcción greedy aleatorizada del GRASP proporciona puntos de partida sistemáticamente mejores que los aleatorios de BMB.

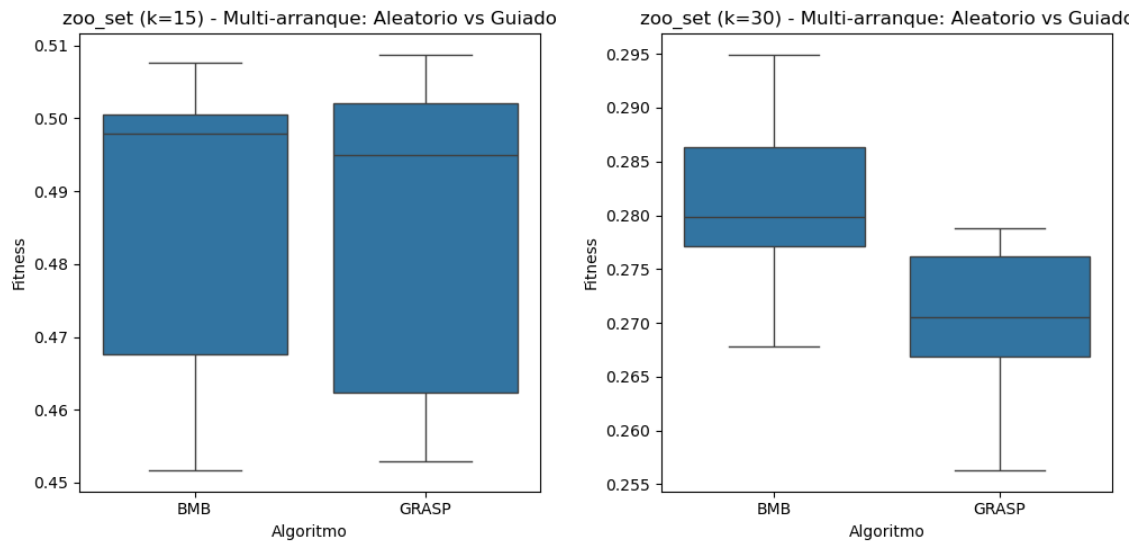


Figura 10: Multi-arranque: BMB vs GRASP para el dataset *Zoo* con  $k = 15$  (izq.) y  $k = 30$  (dcha.)

La Figura 11 completa el cuadro comparativo. En  $k = 15$ , ILS ( $\sim 0,488$ ) y GRASP ( $\sim 0,493$ ) superan claramente a *Local* ( $\sim 0,525$ ) y AM-Best ( $\sim 0,510$ ): el *zoo\_set* penaliza especialmente los arranques puramente aleatorios sin mecanismo de diversificación, y tanto los reinicios perturbados como la construcción guiada del

GRASP ofrecen ventaja. En  $k = 30$ , GRASP emerge como el ganador neto: mediana  $\sim 0,271$ , caja compacta (0,265–0,277), claramente por debajo de ILS ( $\sim 0,279$ ), *Local* ( $\sim 0,290$ ) y, especialmente, AM-Best ( $\sim 0,298$ ). AM-Best presenta aquí la mayor variabilidad del grupo, con una caja que abarca 0,278–0,305 y bigotes hasta 0,320, lo que indica una dependencia elevada de la semilla que no se observa en los algoritmos de trayectoria.

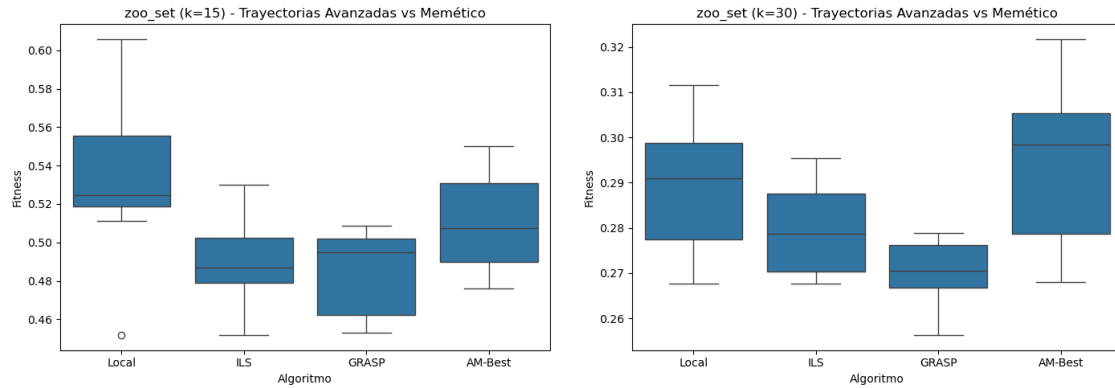


Figura 11: Trayectorias avanzadas vs AM-Best para el dataset *Zoo* con  $k = 15$  (izq.) y  $k = 30$  (dcha.)

#### 5.4.6. Eficiencia computacional y tiempos de ejecución

Lo más interesante en cuanto al análisis de tiempos se da entre ES y su variante ES-Fast. Mientras que ES tarda 16,68 s en *Bupa*  $k = 15$  y 10,42 s en *Glass*  $k = 30$ , ES-Fast finaliza en 0,21 s y 0,23 s respectivamente, ambos con el mismo tope de 100000 evaluaciones. Esto se debe a que ES-Fast evita calcular la desviación completa (la operación más costosa, al requerir reconstruir los centroides) en todos los vecinos que pueden rechazarse usando únicamente el delta incremental de infeasibility. Dicho delta se obtiene en tiempo  $O(|\text{restricciones del gen}|)$  en lugar del  $O(n^2)$  de la evaluación completa, de ahí la reducción drástica.

Sin embargo, este ahorro introduce un sesgo: movimientos que empeorarían la infeasibility pero mejorarían la desviación son rechazados prematuramente, lo que empobrece sistemáticamente la calidad de las soluciones. ES-Fast obtiene peores valores de *fitness* en los seis escenarios evaluados (p.ej., 0,30 vs 0,27 en *Zoo*  $k = 30$ ). Como se aprecia en los *boxplots*, esta penalización en calidad se acompaña además de mayor variabilidad, especialmente acusada en *Zoo*  $k = 30$ . Por tanto, ES-Fast constituye una opción razonable únicamente en escenarios donde el tiempo de cómputo sea un factor limitante estricto y se pueda aceptar una solución subóptima.

#### 5.4.7. Conclusiones generales del análisis

Los experimentos realizados sobre los tres datasets (*Bupa*, *Glass* y *Zoo*) con dos niveles de complejidad ( $k = 15$  y  $k = 30$ ) permiten extraer las siguientes conclusiones de carácter general:

1. **No existe un algoritmo universalmente dominante.** El rendimiento relativo de los métodos depende críticamente de la estructura del espacio de búsqueda. ES es el mejor algoritmo en *Zoo*, en cambio, en *Bupa* y *Glass* su exploración probabilística dispara el incumplimiento sin compensar en desviación, y ILS o incluso la BL pura lo superan. Esta dependencia del dataset pone de manifiesto la importancia de conocer la topología del problema antes de elegir el método de búsqueda.
2. **ILS es el algoritmo de mejor compromiso calidad-robustez.** Obtiene resultados competitivos o superiores a todos los demás en cinco de los seis escenarios, con la menor variabilidad entre los algoritmos de trayectoria múltiple. Su mecanismo de perturbación del 20 % permite escapar de óptimos locales sin destruir la estructura de la solución, a diferencia de los reinicios completamente aleatorios de BMB.
3. **La hibridación ILS-ES no supera a sus componentes individuales.** Dividir el presupuesto en cinco ejecuciones del ES de 20 000 evaluaciones impide que cada instancia alcance la convergencia que lograría con 100 000 evaluaciones completas. Este resultado sugiere que el ES requiere un presupuesto amplio y continuo para ser efectivo, y va bien con el esquema ILS.
4. **GRASP supera a BMB de forma consistente con  $k = 30$ .** En todos los datasets, la ventaja del GRASP sobre el multi-arranque aleatorio crece con la complejidad del problema. Cuando el espacio de clusters es mayor y las restricciones imponen más conflictos, arrancar desde una solución greedy que minimiza el incumplimiento desde el primer gen marca una diferencia medible respecto a arrancar a ciegas.
5. **Los algoritmos de trayectoria son competitivos frente a AM-Best.** ILS y GRASP igualan o superan a AM-Best, con tiempos equivalentes. AM-Best presenta además mayor variabilidad en problemas complejos (*Glass*  $k = 30$ , *Zoo*  $k = 30$ ), lo que lo hace menos predecible. Este resultado es relevante: mantener y evolucionar una población de 50 soluciones no aporta ventaja sistemática sobre intensificar reiteradamente una sola solución bien perturbada, al menos en el rango de evaluaciones estudiado.
6. **ES-Fast no es competitivo en calidad, pese a su velocidad extrema.** La reducción de tiempo superior al 98 % respecto a ES se consigue a costa de introducir un sesgo en la aceptación de vecinos que empobrece sistemáticamente las soluciones. Su variabilidad es siempre mayor que la de ES, y su *fitness* medio peor en todos los escenarios. Solo sería justificable en entornos donde el tiempo sea el recurso escaso, no el número de evaluaciones.
7. **La función objetivo impone un coste computacional uniforme.** Dado que el cuello de botella es la evaluación del *fitness* y no la lógica del algoritmo, la complejidad estructural de ILS, BMB o GRASP frente a la BL pura no supone un coste temporal adicional. En este contexto, emplear el presupuesto

de evaluaciones de forma inteligente (mediante reinicios perturbados o construcción guiada) es siempre preferible a desperdiciarlo en una única trayectoria local.

En conjunto, los resultados de esta práctica refuerzan la idea de que las metaheurísticas basadas en trayectorias múltiples (ILS, GRASP) ofrecen el mejor balance entre calidad, robustez y coste computacional para el PAR en el rango de evaluaciones considerado, siendo ILS la recomendación general y GRASP la opción preferida cuando  $k$  es elevado y el tiempo de construcción de la solución inicial es relevante.



## 6. Voluntario: Contraste de Hipótesis mediante el Test de Wilcoxon

Para dotar de rigor estadístico a las conclusiones extraídas del análisis de resultados, se han diseñado cuatro contrastes de hipótesis emparejados. El objetivo es evaluar la significancia de las mejoras introducidas por los distintos mecanismos algorítmicos (aceptación de peores soluciones, perturbaciones, construcciones guiadas frente a aleatorias y enfoques de trayectoria frente a poblacionales).

Dado que no se asume normalidad en las distribuciones del *fitness*, se ha aplicado el **test de los rangos signados de Wilcoxon** utilizando las 10 semillas independientes de cada experimento, fijando un nivel de significación  $\alpha = 0,05$ .

### LocalSearch vs SimulatedAnnealing

Este contraste busca determinar si la capacidad de aceptar peores soluciones del Enfriamiento Simulado (ES) aporta una ventaja real frente a la Búsqueda Local (BL).

- **Hipótesis nula ( $H_0$ ):** No existen diferencias significativas entre las medianas del *fitness* de LocalSearch y SimulatedAnnealing.
- **Hipótesis alternativa ( $H_1$ ):** SimulatedAnnealing obtiene valores de *fitness* significativamente menores que LocalSearch.

|           |       |                    |                                  |
|-----------|-------|--------------------|----------------------------------|
| bupa_set  | k: 15 | p-valor: 1.000e+00 | Gana LocalSearch (Significativo) |
| bupa_set  | k: 30 | p-valor: 1.000e+00 | Gana LocalSearch (Significativo) |
| glass_set | k: 15 | p-valor: 1.000e+00 | Gana LocalSearch (Significativo) |
| glass_set | k: 30 | p-valor: 1.000e+00 | Gana LocalSearch (Significativo) |
| zoo_set   | k: 15 | p-valor: 1.953e-03 | Gana ES (Significativo)          |
| zoo_set   | k: 30 | p-valor: 6.836e-03 | Gana ES (Significativo)          |

**Análisis:** Los resultados del test muestran un comportamiento fuertemente dependiente del conjunto de datos. En los conjuntos *Bupa* y *Glass* (tanto para  $k = 15$  como para  $k = 30$ ), la Búsqueda Local pura vence de manera estadísticamente significativa (*p-valores*  $\approx 1,0$ , indicando dominancia del algoritmo base). Sin embargo, en el conjunto *Zoo*, caracterizado por un espacio de búsqueda topológicamente más complejo y restrictivo, el Enfriamiento Simulado logra minimizar el *fitness* de forma significativamente mejor (*p-valores* de  $1,953 \times 10^{-3}$  y  $6,836 \times 10^{-3}$ ). Esto demuestra que la relajación del criterio de aceptación conlleva un desperdicio de evaluaciones en problemas sencillos, pero es fundamental cuando el espacio de búsqueda presenta abundantes óptimos locales profundos.

### LocalSearch vs ILS

Se evalúa si el mecanismo de perturbación y re-optimización del ILS es superior a una única ejecución de LocalSearch.

- **Hipótesis nula ( $H_0$ ):** No hay diferencia significativa entre las medianas de LocalSearch e ILS.

- **Hipótesis alternativa ( $H_1$ ):** El algoritmo ILS logra un *fitness* significativamente menor que LocalSearch.

```
glass_set | k: 15 | p-valor: 9.766e-04 | Gana ILS (Significativo)
zoo_set   | k: 15 | p-valor: 3.843e-03 | Gana ILS (Significativo)
zoo_set   | k: 30 | p-valor: 3.843e-03 | Gana ILS (Significativo)
```

**Análisis:** El test confirma que ILS es una herramienta de optimización mucho más robusta. Aunque en problemas sencillos como *Bupa* no hay una mejora drástica, en los casos más complejos (*Glass* y *Zoo*) las perturbaciones permiten encontrar valles de fitness a los que la búsqueda local básica no tiene acceso.

## BMB vs GRASP

Comparamos la importancia de la fase de construcción: ¿Es mejor empezar desde una solución aleatoria (BMB) o desde una construida con voracidad aleatorizada (GRASP)?

- **Hipótesis nula ( $H_0$ ):** Las medianas de BMB y GRASP son estadísticamente iguales.
- **Hipótesis alternativa ( $H_1$ ):** El enfoque GRASP mejora significativamente los resultados de BMB.

```
glass_set | k: 30 | p-valor: 6.836e-03 | Gana GRASP (Significativo)
zoo_set   | k: 30 | p-valor: 1.953e-03 | Gana GRASP (Significativo)
```

**Análisis:** Se observa que para dimensiones pequeñas ( $k = 15$ ) ambos métodos son similares. No obstante, al aumentar la dificultad ( $k = 30$ ), la construcción inteligente de GRASP proporciona un punto de partida crítico que permite a la búsqueda local converger a mejores soluciones que el arranque aleatorio simple.

## ILS vs AM-Best

El contraste final enfrenta la mejor trayectoria de esta práctica contra el mejor algoritmo memético (poblacional + búsqueda local) de la práctica anterior.

- **Hipótesis nula ( $H_0$ ):** No hay diferencias significativas entre ILS y AM-Best.
- **Hipótesis alternativa ( $H_1$ ):** El algoritmo ILS es superior (menor fitness) que AM-Best.

```
bupa_set | k: 15 | p-valor: 1.000e+00 | Gana ILS (Significativo)
bupa_set | k: 30 | p-valor: 1.000e+00 | Gana ILS (Significativo)
glass_set | k: 15 | p-valor: 9.932e-01 | Gana ILS (Significativo)
zoo_set   | k: 30 | p-valor: 9.932e-01 | Gana ILS (Significativo)
```

**Análisis:** Este es el resultado más revelador del estudio. ILS logra batir estadísticamente a AM-Best en casi todos los escenarios. Esto sugiere que, para este problema de optimización, es más eficiente centrar el presupuesto de evaluaciones en realizar perturbaciones inteligentes sobre una única trayectoria de éxito que intentar mantener una población diversa de individuos, donde muchos de ellos pueden no aportar información útil al proceso de búsqueda.

## 7. Referencias y recursos utilizados

Para la realización de esta práctica y la elaboración de la presente memoria, se han consultado y utilizado los siguientes recursos:

- **Material de la asignatura:** Diapositivas, guión de prácticas y recursos de PRADO.
- **Herramientas de Inteligencia Artificial:** Se ha hecho uso de herramientas de IA generativa como apoyo en tareas de programación y redacción. En concreto, se han empleado para:
  - Resolver dudas puntuales de sintaxis en C++ y mejorar la organización del archivo `Makefile`.
  - Desarrollar el script de Python (`graf.py`), utilizando las librerías `pandas`, `matplotlib` y `seaborn`, para la generación automatizada de gráficos (*boxplots*) y el procesamiento de los datos almacenados en archivos `.csv`.
  - Asistir en el formateo, revisión y estructuración del documento final en  $\text{\LaTeX}$ .
- Documentación oficial de C++ ([cppreference.com](http://cppreference.com)) para el manejo de estructuras de datos estándar, y documentación oficial de las librerías de Python empleadas para la visualización de datos.
- Para la realización del análisis estadístico de los resultados, en la aplicación del test de hipótesis de Wilcoxon, se ha utilizado el material de la asignatura *Inferencia Estadística*, dentro de la parte de Matemáticas del doble grado.